Tarea 3: Implementación de la BD de Spotify

---

1. Objetivo

Implementar en **PostgreSQL** la base de datos transaccional de Spotify a partir de tu diagrama ER de la Tarea 2, cargarla con los datos de Laura y comprobar que sus reglas de negocio se cumplen.

Tu repositorio de esta tarea es el **punto de partida del resto del curso**: las siguientes prácticas se construyen sobre él.

---

2. Conceptos que vas a usar

2.1 Archivos `.sql`

Un archivo `.sql` es un archivo de texto con instrucciones de SQL, separadas por punto y coma. Se escribe en cualquier editor. El archivo no hace nada por sí solo: hay que enviárselo a PostgreSQL, que ejecuta sus instrucciones de arriba hacia abajo. Guardar el SQL en archivos, y no solo teclearlo en una consola, permite versionarlo en Git y volver a crear las tablas y los datos cuantas veces haga falta.

Las líneas que empiezan con `--` son comentarios.

2.2 El esquema (`schema.sql`)

El **esquema** es la estructura de una base de datos: sus tablas, columnas, tipos de dato y restricciones. No incluye los datos. El archivo `schema.sql` contiene las sentencias `CREATE TABLE` que crean esas tablas a partir de lo que definiste en tu diagrama ER. La base de datos debe existir antes de ejecutarlo: `CREATE TABLE` crea tablas, no bases de datos.

Por ejemplo, con la clínica veterinaria:

```
CREATE TABLE owners (
    owner_id integer      PRIMARY KEY,
    email    varchar(255) NOT NULL UNIQUE,
    phone    varchar(20)  NOT NULL
);

CREATE TABLE pets (
    pet_id   integer     PRIMARY KEY,
    name     varchar(50) NOT NULL,
    species  varchar(20) NOT NULL,
    owner_id integer     NOT NULL REFERENCES owners (owner_id)
);
```

El orden importa: `owners` va primero porque `pets` hace referencia a ella. El `NOT NULL` de `owner_id` es el mínimo de 1 de la relación: toda mascota tiene dueño.

2.3 Los datos iniciales (`seed.sql`)

El **seed** (semilla) es el archivo con las sentencias `INSERT` que agregan los datos iniciales a las tablas. Se ejecuta después del esquema, porque las tablas ya deben existir. También aquí importa el orden: primero el dueño y después su mascota.

```
INSERT INTO owners (owner_id, email, phone) VALUES (1, 'carla@mail.com',
'6621234567');
INSERT INTO pets (pet_id, name, species, owner_id) VALUES (1, 'Luna', 'perro', 1);
```

2.4 Docker y Docker Compose

Docker ejecuta programas dentro de **contenedores**: entornos aislados que traen todo lo que el programa necesita. Así, todos usamos la misma versión de PostgreSQL sin instalarla directamente en la computadora.

Una imagen es el paquete a partir del cual se crea un contenedor: contiene el programa ya instalado y todo lo que necesita para correr. La imagen es lo que se descarga y el contenedor es lo que corre. Docker descarga cada imagen una sola vez y la guarda en tu computadora. Lo que ocurre dentro de un contenedor no modifica su imagen, y eliminar el contenedor tampoco la borra, así que se puede crear otro a partir de ella sin volver a descargarla.

El nombre de una imagen indica el programa y su versión: `postgres:18` es la imagen de PostgreSQL 18. Además de PostgreSQL, esta imagen trae `psql` y las instrucciones que preparan la base de datos la primera vez que inicia (ver la sección 2.6).

Un volumen es un espacio de almacenamiento que administra Docker, fuera de los contenedores. Hace falta porque lo que se escribe dentro de un contenedor se pierde al eliminarlo. Un contenedor se conecta a un volumen en una ruta, y lo que guarda en esa ruta queda en el volumen. Así, los contenedores se pueden eliminar y volver a crear, y los datos siguen ahí hasta que se borre el volumen.

Docker Compose es una herramienta de Docker que crea e inicia contenedores siguiendo la configuración de un archivo, `docker-compose.yml`, para no tener que escribirla en un comando largo. **Docker y Docker Compose no saben nada de bases de datos:** solo ponen a correr el contenedor. Quien maneja las bases de datos es PostgreSQL, el programa que corre dentro de él.

Un servicio es la configuración con la que Docker Compose crea un contenedor, escrita en `docker-compose.yml`: qué imagen usa, qué variables recibe y qué archivos ve. **El servicio es la descripción y el contenedor es lo que corre a partir de ella.** Aquí tenemos un servicio, al que llamamos `postgres`. Al ejecutar `docker compose up`, Docker Compose crea un contenedor para ese servicio y le pone un nombre con el formato `<carpeta>-<servicio>-<número>`; por ejemplo, en la carpeta `mi-tarea` se llamaría `mi-tarea-postgres-1`.

Si eliminas el contenedor, el servicio sigue descrito en el archivo, y Docker Compose puede crear otro contenedor a partir de él. En los comandos de `docker compose` usas el nombre del servicio, y Docker Compose encuentra su contenedor. Ese nombre más largo es el que verás al ejecutar `docker compose ps`.

Este es el archivo del repositorio base:

```
services:
  postgres:                 # Nombre que le damos a este servicio
    image: postgres:18      # Imagen del contenedor: trae PostgreSQL 18 y psql
    environment:            # Usuario, contraseña y nombre de la base de datos a
crear, tomados del .env
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    ports:
      - "5432:5432"         # Conecta el puerto 5432 de tu computadora con el de
PostgreSQL en el contenedor
    volumes:
      - ./db:/docker-entrypoint-initdb.d:ro   # El contenedor ve tu carpeta db/ en
esta ruta, de donde toma los archivos a ejecutar
      - ./tests.sql:/tests.sql:ro             # El contenedor ve tu tests.sql,
para poder ejecutarlo con psql
      - datos:/var/lib/postgresql             # Volumen: guarda los datos fuera
del contenedor, aunque este se elimine

volumes:
  datos:
```

Elemento	Qué significa
`services:`	Agrupa los servicios que Docker Compose va a ejecutar. Cada servicio indica cómo crear un contenedor: con qué imagen, variables, puertos y volúmenes. Aquí solo hay un servicio, así que Docker Compose crea un solo contenedor
`postgres:`	El nombre del servicio. Lo elegimos nosotros y es el que usan comandos como `docker compose logs postgres`
`image: postgres:18`	La imagen a partir de la cual se crea el contenedor (el primer `postgres` es el nombre de la imagen, no el del servicio)
`environment:`	Las variables de entorno que recibe el contenedor. La imagen de PostgreSQL las lee para saber qué usuario, contraseña y base de datos crear
`ports:`	Conecta un puerto de tu computadora con uno del contenedor, con el formato `computadora:contenedor`. Un contenedor está aislado: sin esto, los programas de tu computadora no podrían conectarse a PostgreSQL
`volumes:` dentro de `postgres:`	Las carpetas y archivos que el contenedor puede ver, con el formato `origen:ruta-en-el-contenedor` (se explican abajo)
`volumes:` al final, con `datos:`	Crea el volumen `datos`, si no existe. Va fuera del servicio porque el volumen no le pertenece al contenedor: debe seguir existiendo cuando el contenedor se elimine (se explica abajo)

Por qué `datos` aparece dos veces. No son dos volúmenes: es uno solo, y cada mención responde una pregunta distinta:

Dónde aparece	Qué pregunta responde	Qué hace
`volumes:` al final, fuera del servicio	¿Qué volúmenes existen?	Le indica a Docker Compose que debe crear el volumen `datos`, si no existe
`datos:/var/lib/postgresql`, dentro del servicio	¿Qué contenedor lo usa, y en qué ruta?	Conecta ese volumen con la ruta `/var/lib/postgresql` del contenedor. No lo crea: si `datos` no estuviera declarado afuera, `docker compose up` terminaría con un error

Se declara fuera del servicio porque su vida es independiente de la del contenedor. Lo que está dentro del servicio describe al contenedor, y este se elimina con `docker compose down`. El volumen no: sobrevive, y el siguiente contenedor se conecta a él y encuentra ahí tus tablas y datos.

Por qué Docker Compose exige declararlo. Docker Compose revisa lo que va antes de los dos puntos en cada línea de `volumes:` del servicio. Si empieza con `./`, como `./db`, es una ruta de tu computadora y no requiere declaración. Si es solo un nombre, como `datos`, es un volumen, y debe aparecer en la sección `volumes:` del final. Si no aparece, Docker Compose considera inválido el archivo y se detiene antes de crear nada:

```
service "postgres" refers to undefined volume datos: invalid compose project
```

Gracias a esta regla, un error de dedo no pasa inadvertido. Si escribieras `dato:/var/lib/postgresql` y Docker Compose lo aceptara, PostgreSQL iniciaría con un volumen nuevo y vacío, y parecería que perdiste tus tablas.

La sección `volumes:` del servicio tiene tres líneas, y hacen cosas distintas. Las dos primeras no usan un volumen de Docker: comparten con el contenedor algo que ya está en tu carpeta, y la tercera conecta el volumen `datos`:

Línea	Origen	Dónde lo ve el contenedor	Para qué
`./db:/docker-entrypoint-initdb.d:ro`	Tu carpeta `db/`	`/docker-entrypoint-initdb.d`	Es la ruta donde la imagen de PostgreSQL busca los archivos que debe ejecutar al inicializarse
`./tests.sql:/tests.sql:ro`	Tu archivo `tests.sql`	`/tests.sql`	Sin esto, `psql` no podría abrirlo, porque corre dentro del contenedor y no ve tus archivos
`datos:/var/lib/postgresql`	El volumen `datos`	`/var/lib/postgresql`	Es donde PostgreSQL guarda las bases de

Línea	Origen	Dónde lo ve el contenedor	Para qué
			datos. Así, tus tablas y datos no se pierden al eliminar el contenedor

Qué datos guarda el volumen `datos`. Todo lo que PostgreSQL necesita conservar: el usuario y la base de datos que se crean al inicializarse, tus tablas, las filas que insertas, y su configuración. Es decir, el resultado de ejecutar tus archivos, no los archivos: `01_schema.sql` y `02_seed.sql` siguen en tu carpeta `db/`.

Por qué esa ruta. No la elegimos nosotros: PostgreSQL guarda todo eso en una carpeta llamada *directorio de datos*, y la imagen `postgres:18` la ubica en `/var/lib/postgresql/18/docker`. Conectar el volumen en `/var/lib/postgresql` hace que esa carpeta quede dentro del volumen. Por eso PostgreSQL no necesita saber que existe un volumen: escribe en su ruta de siempre, y lo que escribe queda fuera del contenedor. Si conectaras `datos` a otra ruta, PostgreSQL seguiría escribiendo en la suya, y tus tablas no quedarían en `datos`.

`./` significa «en la misma carpeta que `docker-compose.yml`». `:ro` significa *read only*: el contenedor puede leer esos archivos, pero no modificarlos. Si editas uno en tu carpeta, el contenedor ve el cambio, porque es el mismo archivo y no una copia.

Si tu editor guarda reemplazando el archivo, el contenedor puede seguir viendo el `tests.sql` anterior; se resuelve con `docker compose down` y `docker compose up -d`.

2.5 Credenciales y el archivo `.env`

Las contraseñas y otras credenciales no se escriben directamente en el código: cualquiera con acceso al repositorio las vería, y quedarían para siempre en el historial de Git. Por eso se guardan en un archivo aparte llamado `.env`, con una variable por línea:

```
POSTGRES_USER=admin
POSTGRES_PASSWORD=vet_pass_123
POSTGRES_DB=veterinaria
```

Docker Compose lee automáticamente el `.env` que está en la misma carpeta que `docker-compose.yml`, y sustituye cada `${VARIABLE}` por su valor:

```
environment:
  POSTGRES_USER: ${POSTGRES_USER}          # Se convierte en: admin
  POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
  POSTGRES_DB: ${POSTGRES_DB}
```

> ⚠ **EL ARCHIVO `.env` NO SE SUBE AL REPOSITORIO, NUNCA.** Revisa que no aparezca en `git status` antes de hacer un commit. Una contraseña que llega a GitHub se considera comprometida, aunque después borres el archivo.

Para evitarlo, el repositorio base incluye dos archivos:

- `.gitignore`**:** le indica a Git qué archivos ignorar. Ya incluye `.env`.
- `.env.example`**:** una plantilla con los nombres de las variables, que sí se sube, para que quien clone el repositorio sepa qué debe configurar.

Cada persona crea su propio `.env` a partir de la plantilla:

```
cp .env.example .env
```

En esta tarea, la contraseña es de una base de datos local de práctica, pero el hábito es el mismo que en un sistema real.

Estos valores solo se usan cuando PostgreSQL se inicializa (ver la sección 2.6). Si los cambias después, no tendrán efecto hasta que borres el volumen.

2.6 Qué hace cada pieza

Al ejecutar `docker compose up -d` por primera vez, ocurre esta cadena. Cada paso lo hace una pieza distinta:

#	Quién lo hace	Qué hace
1	**Docker Compose**	Lee `docker-compose.yml` y `.env`, y le pide a Docker crear e iniciar el contenedor
2	**Docker**	Descarga la imagen `postgres:18`, si no la tienes, y crea e inicia el contenedor
3	**La imagen de PostgreSQL**, al iniciar	Si el volumen está vacío, prepara el directorio de datos de PostgreSQL, crea el usuario y **crea la base de datos** con los nombres del `.env`
4	**PostgreSQL**, con tu `db/01_schema.sql`	**Crea las tablas** dentro de esa base de datos
5	**PostgreSQL**, con tu `db/02_seed.sql`	**Inserta los datos** en esas tablas
6	**PostgreSQL**	Queda corriendo dentro del contenedor, listo para recibir conexiones

A partir de la segunda vez, el volumen ya tiene datos, así que **los pasos 3 a 5 no se repiten**: PostgreSQL simplemente arranca con lo que ya estaba guardado.

Para enviarle instrucciones a PostgreSQL se usa `psql`, un programa cliente de línea de comandos que también viene en la imagen.

---

3. El repositorio base

El repositorio base es un **ejemplo completo y funcional**: implementa la clínica veterinaria de la presentación. Lo clonas para comprobar que tu instalación funciona (sección 5.4) y para guiarte al hacer tu tarea.

Tu tarea no se hace en el repositorio base. Va en tu propio repositorio, creado desde cero e independiente: no es un fork ni una rama del repositorio base.

El repositorio base incluye:

- `docker-compose.yml`: la configuración del contenedor de PostgreSQL.
- `.env.example` y `.gitignore`: la plantilla de las credenciales y la protección para no subirlas.
- `db/01_schema.sql` y `db/02_seed.sql`: las sentencias que crean las tablas e insertan los datos. Se ejecutan en orden alfabético, solo cuando PostgreSQL se inicializa con un volumen vacío (ver la sección 2.6); por eso llevan los prefijos `01_` y `02_`.
- `tests.sql`: las consultas y pruebas, que se ejecutan con `psql`.
- `docs/`: la carpeta de la documentación de la base de datos.

---

4. Lo que hay que implementar

4.1 Esquema (`db/01_schema.sql`)

Crea todas las tablas de tu diagrama ER con sus tipos de dato, llaves primarias y foráneas. **Cada cardinalidad debe reflejarse en una restricción** cuando sea posible: piensa qué implica cada mínimo y cada máximo para `NOT NULL`, `UNIQUE` y las llaves foráneas.

4.2 Datos (`db/02_seed.sql`)

Inserta **todos los datos de las tablas A, B, C y D de Laura**, conservando sus identificadores (`song_id`, `album_id`, `artist_id`, `playlist_id`).

Agrega los datos que falten para que existan estos casos:

- Una suscripción sin suscriptores
- Un artista sin canciones ni álbumes
- Una playlist vacía
- Una canción sin álbum

4.3 Pruebas (`tests.sql`)

a) Las preguntas de Laura. Una consulta por pregunta:

> **Laura:** Ya que tengan la base, necesito saber:
>
> 1. ¿Qué canciones tiene la playlist 10, en orden, y quiénes son sus artistas?
> 2. ¿Cuántos suscriptores tiene cada suscripción? Que aparezcan también las que no tienen ninguno.
> 3. ¿Qué canciones no están en ningún álbum?
> 4. ¿Qué tiene guardado cada suscriptor en su biblioteca, tanto canciones como álbumes?
> 5. ¿Cuál es la canción más escuchada?

Si alguna pregunta no se puede responder con tu base, explica en un comentario qué le falta.

b) Intentos de romper las reglas. Al menos un `INSERT`, `UPDATE` o `DELETE` que PostgreSQL deba rechazar por cada tipo de restricción que usaste, con un comentario sobre la regla de negocio que protege.

c) Registro y eliminación de un suscriptor. Implementa cada operación con una **transacción** (tutorial de PostgreSQL). Ambas deben ser **todo o nada**:

- **Registro:** crea al suscriptor, su perfil y su biblioteca. Nunca puede quedar un suscriptor sin perfil o sin biblioteca.
- **Eliminación:** borra al suscriptor y todo lo que depende de él: su perfil, su biblioteca con lo que tiene guardado, y sus playlists con sus canciones. No debe borrarse ninguna canción, álbum ni artista del catálogo. No uses `ON DELETE CASCADE`: cada borrado debe ser explícito.

Para cada operación, incluye un caso exitoso y uno en el que falle a medio camino, y muestra con una consulta que los datos quedaron como estaban antes de empezar.

---

5. Cómo ejecutar tu código y el repo base

5.1 Qué vas a instalar y por qué

PostgreSQL corre en un contenedor de Docker, Docker corre en Linux y, en Windows, Linux corre dentro de WSL. Por eso cada pieza va dentro de la anterior y **el orden de instalación importa**:

```
Windows
└── WSL 2: permite ejecutar Linux dentro de Windows
    └── Ubuntu: la distribución de Linux; aquí abres tu terminal y trabajas
        └── Docker Engine: ejecuta contenedores
            └── Contenedor con PostgreSQL y psql: no los instalas tú, vienen en la
imagen que descarga Docker
```

Si usas macOS o Linux, no necesitas WSL. En Linux, instala Docker Engine directamente (paso 3). En macOS, usa Docker Desktop. Continúa en el paso 4.

5.2 Instalación

Paso 1. Instala WSL. Abre PowerShell como administrador y ejecuta el siguiente comando. Reinicia si te lo pide.

```
wsl --install --no-distribution
```

Paso 2. Instala Ubuntu. En la misma ventana de PowerShell, ejecuta:

```
wsl --install -d Ubuntu
```

Te pedirá crear un usuario y una contraseña de Linux. Después, abre **Ubuntu** desde el menú de inicio. **A partir de aquí, todos los comandos se ejecutan en la terminal de Ubuntu**, no en PowerShell ni en `cmd`.

Paso 3. Instala Docker Engine dentro de Ubuntu. El script oficial instala Docker y el plugin de Docker Compose:

```
curl -fsSL https://get.docker.com | sudo sh
```

Da permiso a tu usuario para usar Docker sin `sudo`:

```
sudo usermod -aG docker $USER
```

Cierra la terminal de Ubuntu y vuelve a abrirla para que el permiso se aplique.

Paso 4. Comprueba que Docker funciona. El primer comando debe mostrar un mensaje que empieza con «Hello from Docker!», y el segundo, una versión:

```
docker run hello-world
docker compose version
```

Si aparece un error que dice que no se puede conectar con Docker, es que su servicio no está corriendo en Ubuntu. Inícialo con `sudo service docker start` y vuelve a intentarlo.

Paso 5. Instala Git y prepara tu editor. En Ubuntu, ejecuta:

```
sudo apt update
sudo apt install -y git
```

Puedes usar cualquier editor. Con VS Code, instala la extensión **WSL** y abre tu repositorio con `code .` desde la terminal de Ubuntu.

> **Clona tus repositorios dentro de tu carpeta de Linux** (por ejemplo, `~/dataeng`), no en `/mnt/c/`.

5.3 Los comandos de Docker que vas a usar

Ejecútalos desde la carpeta de tu repositorio, donde está `docker-compose.yml`.

Qué quieres hacer	Comando	Qué ocurre
Iniciar PostgreSQL	`docker compose up -d`	Crea e inicia el contenedor; `-d` lo deja corriendo en segundo plano. Si el volumen está vacío, además ocurre la inicialización de la sección 2.6
Ver si el contenedor está corriendo	`docker compose ps`	Lista los contenedores de este proyecto y su estado
Ver los mensajes de PostgreSQL y los errores de tus archivos	`docker compose logs postgres`	Muestra lo que ha escrito el contenedor del servicio `postgres`

Qué quieres hacer	Comando	Qué ocurre
Abrir una consola para escribir SQL (se sale con `\q`)	`docker compose exec postgres psql -U admin -d spotify`	Ejecuta `psql` dentro del contenedor y lo conecta a la base de datos
Ejecutar tus pruebas	`docker compose exec postgres psql -U admin -d spotify -f /tests.sql`	Igual, pero `psql` ejecuta las instrucciones del archivo y termina
Detener PostgreSQL y conservar los datos	`docker compose down`	Detiene y elimina el contenedor; el volumen se queda
Eliminar también todos los datos	`docker compose down -v`	Detiene y elimina el contenedor y **borra el volumen**: se pierden la base de datos, sus tablas y sus datos

El comando de `psql` se explica pieza por pieza en la sección 5.4, paso 4. Sus valores de `-U` y `-d` deben coincidir con el `.env` de la carpeta donde estás: esta tabla usa los de tu tarea (`spotify`); en el repo base, la base de datos es `veterinaria`.

5.4 Cómo correr el repo base

Hazlo **antes de empezar tu tarea**, para comprobar que tu instalación funciona. El repo base trae la clínica veterinaria lista para ejecutarse.

Paso 1. Descarga el repositorio base.

```
git clone https://github.com/Pedro-Hdez/vet-clinic-db.git
cd vet-clinic-db
```

`git clone` crea en tu computadora una carpeta con una copia del repositorio. `cd` te coloca dentro de ella. Es necesario porque Docker Compose busca `docker-compose.yml` y `.env` en la carpeta donde estás.

Paso 2. Crea tu `.env` e inicia PostgreSQL.

```
cp .env.example .env
docker compose up -d
```

`cp` copia la plantilla con el nombre `.env`, que es el archivo que lee Docker Compose. El repositorio no lo trae porque no se sube a Git, y sin él PostgreSQL no tendría usuario, contraseña ni base de datos que crear.

`docker compose up` crea e inicia lo que describe `docker-compose.yml`, y `-d` lo deja corriendo en segundo plano para que recuperes tu terminal. Como el volumen está vacío, ocurre la cadena de la sección 2.6: se crean el usuario y la base de datos, y se ejecutan los archivos de `db/`.

El comando termina antes que esa inicialización, así que espera unos segundos antes de seguir. La primera vez también tarda porque Docker descarga la imagen.

Paso 3. Revisa que la inicialización no haya tenido errores.

```
docker compose logs postgres | grep ERROR
```

Parte	Qué significa
`docker compose logs postgres`	Muestra los mensajes que ha escrito el contenedor del servicio `postgres`, incluidos los de PostgreSQL al ejecutar los archivos de `db/`
`|`	Envía esa salida al siguiente programa, en lugar de mostrarla en pantalla
`grep ERROR`	Muestra solo las líneas que contienen la palabra `ERROR`

Este paso es necesario porque `up -d` no muestra lo que pasa dentro del contenedor: un error en el SQL no aparecería en tu terminal.

Si el comando no muestra nada, no hubo errores. Si muestra algo, corrige el archivo y repite desde `docker compose down -v`, porque la inicialización se detuvo en ese error.

Haz esta revisión antes de ejecutar las pruebas. Después, también aparecerán aquí los errores de los intentos de romper las reglas, que son correctos.

Paso 4. Ejecuta las pruebas.

```
docker compose exec postgres psql -U admin -d veterinaria -f /tests.sql
```

Los archivos `.sql` no hacen nada por sí solos: hay que enviárselos a PostgreSQL, y para eso se usa `psql`. Tampoco tienes `psql` instalado en tu computadora, porque viene en la imagen. Por eso este comando le pide a Docker que lo ejecute dentro del contenedor:

Parte	Qué significa	Quién la interpreta
`docker compose exec`	Ejecuta un programa dentro de un contenedor que ya está corriendo	Docker Compose
`postgres`	En qué contenedor: el del servicio `postgres` de `docker-compose.yml`	Docker Compose
`psql`	El programa que se ejecuta: el cliente de PostgreSQL	El contenedor
`-U admin`	Con qué usuario se conecta (`POSTGRES_USER`)	`psql`
`-d veterinaria`	A qué base de datos se conecta (`POSTGRES_DB`)	`psql`
`-f /tests.sql`	Qué archivo ejecuta. Es una ruta **dentro del contenedor**: existe porque `docker-compose.yml` hace visible ahí tu `tests.sql`	`psql`

Todo lo que va antes de `psql` le dice a Docker dónde ejecutar el programa, y todo lo que va después le dice a `psql` qué hacer.

No te pide contraseña porque `psql` se conecta desde dentro del mismo contenedor, y la imagen de PostgreSQL confía en esas conexiones locales. La contraseña se exige a las conexiones que llegan por la red, como las de un programa en tu computadora a través del puerto 5432.

Compara el resultado con los comentarios de `tests.sql`. Los intentos de romper las reglas deben mostrar un `ERROR`: significa que PostgreSQL rechazó la operación, y `psql` sigue con la siguiente instrucción.

Entra a tu base de datos y explórala

Antes de eliminar nada, entra a ver lo que acabas de crear. Hasta aquí todo pasó dentro del contenedor y solo viste mensajes. Ahora vas a recorrer el camino a mano, una pieza a la vez, y a consultar la base de datos de la clínica.

1. Entra al contenedor. Abre una terminal de Linux (`bash`) dentro del contenedor:

```
docker compose exec postgres bash
```

Tu terminal cambia a algo como `root@eb4520b3893a:/#`. Ya no estás en tu Ubuntu, sino dentro del contenedor: `root` es el usuario y lo que sigue es el identificador del contenedor. Ejecuta `ls /` y `ls /docker-entrypoint-initdb.d`: verás las carpetas de este otro Linux y tus archivos de `db/`, tal como los hace visibles `docker-compose.yml`.

2. Entra a la base de datos. Dentro del contenedor, abre `psql`:

```
psql -U admin -d veterinaria
```

Tu terminal cambia a `veterinaria=#`. Ya no estás hablando con Linux, sino con PostgreSQL: aquí escribes SQL, y **cada instrucción termina con punto y coma**. Los comandos que empiezan con `\` son de `psql`, no de SQL, y no lo llevan.

3. Mira qué hay.

```
\l
\dt
\d pets
```

Comando	Qué muestra
`\l`	Las bases de datos de este servidor, entre ellas `veterinaria`, la que se creó con el valor de tu `.env`
`\dt`	Las tablas de la base de datos: las cinco que creó `01_schema.sql`

Comando	Qué muestra
`\d pets`	La estructura de `pets`: sus columnas, tipos, llave primaria y llaves foráneas

Si la salida se detiene en `:`, presiona `q`.

4. Ejecuta consultas. ¿Qué mascotas hay y quién es su dueño?

```
SELECT p.name AS mascota, p.species AS especie, o.email AS dueno
FROM pets p
JOIN owners o ON o.owner_id = p.owner_id
ORDER BY p.pet_id;
```

```
 mascota | especie |     dueno
---------+---------+----------------
 Luna    | perro   | carla@mail.com
 Michi   | gato    | jorge@mail.com
 Luna    | gato    | jorge@mail.com
 Rocky   | perro   | sofia@mail.com
 Toby    | perro   | sofia@mail.com
(5 rows)
```

¿Cuántas veces se ha aplicado cada vacuna?

```
SELECT va.name AS vacuna, COUNT(vv.visit_id) AS veces_aplicada
FROM vaccines va
LEFT JOIN visit_vaccine vv ON vv.vaccine_id = va.vaccine_id
GROUP BY va.name
ORDER BY veces_aplicada DESC, va.name;
```

```
    vacuna     | veces_aplicada
---------------+----------------
 Rabia         |              3
 Moquillo      |              2
 Parvovirus    |              2
 Triple felina |              1
 Leptospirosis |              0
(5 rows)
```

Ahora agrega una mascota y vuelve a ejecutar la primera consulta: Firulais aparece, porque la base de datos guardó tu cambio.

```
INSERT INTO pets (pet_id, name, species, owner_id) VALUES (6, 'Firulais', 'perro',
```

```
1);
```

Prueba también tus propias consultas.

5. Sal. Se sale de una pieza a la vez, en orden inverso:

```
\q
exit
```

`\q` cierra `psql` y te regresa a la terminal del contenedor. `exit` cierra esa terminal y te regresa a tu Ubuntu. El contenedor sigue corriendo, y tus datos siguen ahí.

Para abrir `psql` directamente, sin pasar por `bash`, usa el comando de la sección 5.3 (`docker compose exec postgres psql -U admin -d veterinaria`).

Cuando termines

Paso 5. Elimina el contenedor y el volumen.

```
docker compose down -v
```

`down` detiene y elimina el contenedor. Es necesario porque, mientras siga corriendo, ocupa el puerto 5432 y el contenedor de tu tarea no podría iniciar. `-v` además borra el volumen con las tablas y los datos de la clínica, que ya no necesitas; si vuelves a ejecutar el ejemplo, se inicializa desde cero. Los archivos del repositorio no se borran: conserva esta carpeta para consultarla mientras haces tu tarea.

5.5 Cómo verificar tu tarea antes de entregarla

Tu tarea debe pasar por el mismo procedimiento de la sección 5.4, con tu repositorio en lugar del repositorio base. Que funcione en tu carpeta de trabajo no basta, porque ahí pueden estar archivos que olvidaste subir o un volumen con tablas de una versión anterior. Por eso hay que probarla como la va a recibir quien la revise:

1. Sube todos tus cambios a GitHub y, en tu carpeta de trabajo, elimina el contenedor y el volumen con `docker compose down -v`.
2. Clona tu repositorio **en otra carpeta** y entra a ella.
3. Crea ahí tu `.env` con los valores de tu proyecto (`POSTGRES_DB=spotify`) y sigue los pasos 2 a 4 de la sección 5.4, sin el `cp` y con `-d spotify` en lugar de `-d veterinaria`.
4. Comprueba que:
    - Tus archivos de `db/` se ejecutan sin errores.
    - Las preguntas de Laura devuelven lo que esperas.
    - Tus intentos de romper las reglas son rechazados.
    - Las transacciones se comportan como todo o nada.
    - `.env` no aparece en tu repositorio de GitHub.

5. En esa carpeta, elimina el contenedor y el volumen con `docker compose down -v`. Después, borra la carpeta.

---

6. Reglas y consideraciones

1. **Tu base de datos debe poder reconstruirse desde cero.** Después de `docker compose down -v` y `docker compose up -d`, las tablas y los datos deben quedar creados solo con tus archivos de `db/`, sin pasos manuales.
2. **Mantén la documentación al día.** Si tu esquema cambió respecto a tu diagrama ER, actualiza los diagramas de `docs/` y explica el cambio en el `README.md`.
3. **Escribe los supuestos que tomaste** en comentarios o en el `README.md`.
4. **Usa la convención de nombres de la Tarea 2.**
5. **No subas tu `.env` al repositorio.** Ninguna credencial debe aparecer en los archivos ni en el historial de Git.

---

7. Entregables

La liga a tu repositorio de GitHub con:

```
tu-repositorio/
├── db/
│   ├── 01_schema.sql
│   └── 02_seed.sql
├── docs/
│   ├── diagrama_conceptual.png   (o PDF)
│   └── diagrama_er.png           (o PDF)
├── .gitignore
├── docker-compose.yml
├── tests.sql
└── README.md                     (el contexto del proyecto, tus supuestos, las
variables que lleva el `.env` y cómo iniciar PostgreSQL y ejecutar tus pruebas)
```

Tu `.env` existe en tu computadora, pero **no debe aparecer en el repositorio**.

- **Diagrama conceptual:** el diagrama unificado, con las entidades y relaciones nuevas que hayan surgido al normalizar.
- **Diagrama ER:** el de la Tarea 2, con atributos y tipos de dato, PK y FK, y actualizado para que coincida con tu esquema.
